

This document is the official prompt engineering guide for Granite 4.0 models. It highlights basic and advanced features that Granite 4.0 models support and demonstrates how to perform common text generation tasks.

Granite 4.0 highlights

- Simplified and standardized chat template design for general, tool-use, and RAG tasks
- Enhanced tool-calling, RAG, and FIM capabilities.
- Native support for JSON outputs.

Chat template

To maximize the performance of Granite 4.0 instruct models, we recommend constructing prompts using libraries such as `transformers`, which automatically apply the models' official chat template. If users choose to author prompts directly in `Jinja`, they should carefully adhere to the template's design conventions.

Chat template design

The elements of this chat template serve the following purpose:

- `<|start_of_role|>`, `<|end_of_role|>`, and `<|end_of_text|>` are the tags for role and prompt control.
 - `<|start_of_role|>` indicates the start of a role.
 - `<|end_of_role|>` indicates the end of the role.
 - `<|end_of_text|>` indicates the end of a single message block. Each message block should be terminated with `<|end_of_text|>` to ensure proper message segmentation.
- `user`, `assistant`, and `system` are the roles the chat template supports.
 - `user` role identifies queries from the user or an external agent.
 - `assistant` role identifies model generations.

- `system` role identifies system messages. This role, in particular, encompasses several design decisions, which we discuss in detail below.
 - Granite 4.0 models chat template do specify a default system prompt; however, users may define a different one. This is an example of a user-defined system prompt:

```
<|start_of_role|>system<|end_of_role|>You are a weather assistant that responds with relevant function calls instead of natural language.<|end_of_text|>
```

- Granite 4.0 models chat template also supports multiple system turns within a single conversation. The chat template will automatically format them, as demonstrated in the following example.

```
<|start_of_role|>system<|end_of_role|>You are a senior software engineer assistant. Your task is to help users write, debug, and optimize code in Python, C++, and JavaScript.<|end_of_text|>
<|start_of_role|>user<|end_of_role|>How do you create a list in Python?<|end_of_text|>
<|start_of_role|>assistant<|end_of_role|>my_list = [1, 2, 3, 4, 5]
<|end_of_text|>
<|start_of_role|>system<|end_of_role|>You have also recently learned TypeScript programming language.<|end_of_text|>
<|start_of_role|>user<|end_of_role|>Please give me the same code in TypeScript<|end_of_text|>
<|start_of_role|>assistant<|end_of_role|>let myList: number[] = [1, 2, 3, 4, 5]<|end_of_text|>
```

- Granite 4.0 chat template automatically includes the list of tools as part of the system prompt when tools are provided.

```
<|start_of_role|>system<|end_of_role|>You are a helpful assistant with access to the following tools. You may call one or more tools to assist with the user query.
```

You are provided with function signatures within `<tools></tools>` XML tags:

```
<tools>
{"type": "function", "function": {"name": "get_current_weather",
"description": "Get the current weather for a specified city.",
"parameters": {"type": "object", "properties": {"city": {"type": "string", "description": "Name of the city"}}}, "required": ["city"]}
{"type": "function", "function": {"name": "get_time",
"description": "Get the current time for a specified location.",
"parameters": {"type": "object", "properties": {"location":
```

```
{"type": "string", "description": "Coordinates of the location"}},  
"required": ["location"]}}}  
</tools>
```

For each tool call, return a json object with function name and arguments within `<tool_call>``</tool_call>` XML tags:

```
<tool_call>  
{  
  "name": <function-name>,  
  "arguments": <args-json-object>  
}  
</tool_call>.
```

If a tool does not exist in the provided list of tools, notify the user that you do not have the ability to fulfill the request.`<|end_of_text|>`

- Granite 4.0 chat template automatically includes the list of documents as part of the system prompt when documents are provided.

```
<|start_of_role|>system<|end_of_role|>You are a helpful assistant  
with access to the following documents. You may use one or more  
documents to assist with the user query.
```

You are given a list of documents within `<documents>``</documents>` XML tags:

```
<documents>  
{  
  "doc_id": 1, "title": "History Document Title", "text": "From the  
early 12th century, French builders developed the Gothic style,  
marked by the use of rib vaults, pointed arches, flying  
buttresses, and large stained glass windows. It was used mainly in  
churches and cathedrals, and continued in use until the 16th  
century in much of Europe. Classic examples of Gothic architecture  
include Chartres Cathedral and Reims Cathedral in France as well  
as Salisbury Cathedral in England. Stained glass became a crucial  
element in the design of churches, which continued to use  
extensive wall-paintings, now almost all lost.", "source": ""}  
{  
  "doc_id": 2, "title": "Gothic Book", "text": "From long time ago,  
French builders developed the Gothic style, marked by the use of  
rib vaults, pointed arches, flying buttresses, and large stained  
glass windows. It was used mainly in churches and cathedrals, and  
continued in use until the 16th century in much of Europe. Classic  
examples of Gothic architecture include Chartres Cathedral and  
Reims Cathedral in France as well as Salisbury Cathedral in  
England. Stained glass became a crucial element in the design of  
churches, which continued to use extensive wall-paintings, now  
almost all lost.", "source": ""}  
{  
  "doc_id": 3, "title": "More Recent Book", "text": "From  
yesterday, French builders developed the Gothic style, marked by  
the use of rib vaults, pointed arches, flying buttresses, and  
large stained glass windows. It was used mainly in churches and  
cathedrals, and continued in use until the 16th century in much of  
Europe. Classic examples of Gothic architecture include Chartres
```

```
Cathedral and Reims Cathedral in France as well as Salisbury
Cathedral in England. Stained glass became a crucial element in
the design of churches, which continued to use extensive wall-
paintings, now almost all lost.", "source": ""}
</documents>
```

Write the response to the user's input by strictly aligning with the facts in the provided documents. If the information needed to answer the question is not available in the documents, inform the user that the question cannot be answered based on the available data.<|end_of_text|>

- Granite 4.0 chat template also supports combining tool-use and RAG capabilities through an appropriately designed system prompt. Note that order matters, the chat template lists tools first followed by documents. Moreover, if a user-defined system prompt is provided, it will appear first, followed by the tools section and then the documents section.

```
<|start_of_role|>system<|end_of_role|>You are a weather assistant
that responds with relevant function calls instead of natural
language.
```

You are a helpful assistant with access to the following tools.
You may call one or more tools to assist with the user query.

You are provided with function signatures within <tools></tools>
XML tags:

```
<tools>
{"type": "function", "function": {"name": "get_current_weather",
"description": "Get the current weather for a specified city.",
"parameters": {"type": "object", "properties": {"city": {"type":
"string", "description": "Name of the city"}}}, "required":
["city"]}}}
{"type": "function", "function": {"name": "get_time",
"description": "Get the current time for a specified location.",
"parameters": {"type": "object", "properties": {"location":
{"type": "string", "description": "Coordinates of the location"}},
"required": ["location"]}}}
</tools>
```

For each tool call, return a json object with function name and arguments within <tool_call></tool_call> XML tags:

```
<tool_call>
{"name": <function-name>, "arguments": <args-json-object>}
</tool_call>. If a tool does not exist in the provided list of
tools, notify the user that you do not have the ability to fulfill
the request.
```

You are a helpful assistant with access to the following documents. You may use one or more documents to assist with the user query.

You are given a list of documents within <documents></documents> XML tags:

<documents>

{"doc_id": 1, "title": "", "text": "From the early 12th century, French builders developed the Gothic style, marked by the use of rib vaults, pointed arches, flying buttresses, and large stained glass windows. It was used mainly in churches and cathedrals, and continued in use until the 16th century in much of Europe. Classic examples of Gothic architecture include Chartres Cathedral and Reims Cathedral in France as well as Salisbury Cathedral in England. Stained glass became a crucial element in the design of churches, which continued to use extensive wall-paintings, now almost all lost.", "source": ""}

{"doc_id": 2, "title": "", "text": "From long time ago, French builders developed the Gothic style, marked by the use of rib vaults, pointed arches, flying buttresses, and large stained glass windows. It was used mainly in churches and cathedrals, and continued in use until the 16th century in much of Europe. Classic examples of Gothic architecture include Chartres Cathedral and Reims Cathedral in France as well as Salisbury Cathedral in England. Stained glass became a crucial element in the design of churches, which continued to use extensive wall-paintings, now almost all lost.", "source": ""}

{"doc_id": 3, "title": "", "text": "From yesterday, French builders developed the Gothic style, marked by the use of rib vaults, pointed arches, flying buttresses, and large stained glass windows. It was used mainly in churches and cathedrals, and continued in use until the 16th century in much of Europe. Classic examples of Gothic architecture include Chartres Cathedral and Reims Cathedral in France as well as Salisbury Cathedral in England. Stained glass became a crucial element in the design of churches, which continued to use extensive wall-paintings, now almost all lost.", "source": ""}

</documents>

Write the response to the user's input by strictly aligning with the facts in the provided documents. If the information needed to answer the question is not available in the documents, inform the user that the question cannot be answered based on the available data.<|end_of_text|>

- About tool calling

- Granite 4.0 chat template automatically lists tools between `<tools>` and `</tools>` tags as part of the `system` message when a list of tools is provided.
- Granite 4.0 chat template automatically returns tool calls between `<tool_call>` and `</tool_call>` tags within the assistant turn. Example:

```
<|start_of_role|>assistant<|end_of_role|>
<tool_call>
{"name": "get_current_weather", "arguments": {"city": "Bengaluru"}}
</tool_call>
<|end_of_text|>
```

- Moreover, Granite 4.0 chat template converts `tool` role content into a `user` role in which tool responses appear between `<tool_response>` and `</tool_response>` tags. Example:

```
<|start_of_role|>user<|end_of_role|>
<tool_response>
Bengaluru is sunny with a temperature of 30°C.
</tool_response>
<|end_of_text|>
```

- About RAG

- The chat template lists documents as part of the `system` turn between `<documents>` and `</documents>` tags.

- About FIM

- The tags supported for fill-in-the-middle (FIM) code completion are: `<|fim_prefix|>`, `<|fim_middle|>`, and `<|fim_suffix|>`. Make sure to use the correct FIM tags when using Granite 4.0 models for FIM code completions.

i NOTE

- Granite 4.0 models chat template uses reverse iteration over messages to identify the last user query index.
 - All outputs should be terminated with `<|end_of_text|>` to clearly segment messages.

Basic chat template example

Below, we show a basic example of Granite 4.0 models chat template.

```
<|start_of_role|>user<|end_of_role|>What is the largest ocean on Earth?
<|end_of_text|>
<|start_of_role|>assistant<|end_of_role|>The largest ocean on Earth is the
Pacific Ocean. It covers an area of about 63.8 million square miles
(165.25 million square kilometers), which is more than twice the size of
the second-largest ocean, the Atlantic Ocean. The Pacific Ocean lies
between the Americas to the east and Asia and Australia to the west.
<|end_of_text|>
```

Use the following code example to reproduce this output.

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

device = "cuda"
model_path = "ibm-granite/granite-4.0-h-tiny"
tokenizer = AutoTokenizer.from_pretrained(model_path)

# drop device_map if running on CPU
model = AutoModelForCausalLM.from_pretrained(model_path,
device_map=device)
model.eval()

# change input text as desired
chat = [
    { "role": "user", "content": "What is the largest ocean on Earth?"},
]

chat = tokenizer.apply_chat_template(chat, tokenize=False,
add_generation_prompt=True)

# tokenize the text
input_tokens = tokenizer(chat, return_tensors="pt").to(device)
# generate output tokens
output = model.generate(*input_tokens,
                        max_new_tokens=150)
# decode output tokens into text
output = tokenizer.batch_decode(output)

print(output[0])
```

Granite 4.0 inference examples

Basic examples

In this section, we provide basic examples for a variety of inference tasks.

Summarization

This example demonstrates how to summarize an interview transcript.

```
<|start_of_role|>user<|end_of_role|>Summarize a fragment of an interview transcript. In this interview, an NBC reporter interviews Simone Biles about her participation in Paris 2024 Olympic games. Your response should only include the answer. Do not provide any further explanation.
```

```
Speaker 1 (00:00):  
Simone, congratulations.
```

```
Simone (00:02):  
Thank you.
```

```
Speaker 1 (00:02):  
The last several years have been all about our expectations, what we thought you would do, what we thought the team would do. What were your expectations coming here?
```

```
Simone (00:10):  
My expectations or at least the team's expectation was to go out there and win gold for Team USA, and we did just that. And after that, my expectation was go to all-around, see if you can place top three, see what happens. And then after that, everything else was a cherry on top. So Vault, gold. Floor, silver. I'm not mad.
```

```
Speaker 1 (00:28):  
Are you proud of yourself?
```

```
Simone (00:30):  
Yes. I'm very proud of myself for how I held it together all of those competition days, all throughout those routines. It's very rigorous for however many days we compete in a row. So I'm very proud of my performances.
```

```
Speaker 1 (00:41):  
And you've been all about self-care.
```

```
Simone (00:42):  
Yes.
```

```
Speaker 1 (00:43):  
Did it work?
```

Simone (00:44):

Yes. No, most definitely. I've been seeing my therapist a lot while I've been here in Paris, so I owe a lot to her.

Speaker 1 (00:53):

I don't mean to make you do my job, but if you had to write the headlines of these games telling your story, what would it be?

Simone (01:02):

I would say my success is defined by what I make it.

Speaker 1 (01:09):

I like that.

Simone (01:09):

Yeah.

Speaker 1 (01:10):

I like that. I might use that.

Simone (01:11):

Okay. Good.

Speaker 1 (01:13):

You also inspired a lot of people along the way. Have you ever sat back and really thought about what you have contributed to this sport?

Simone (01:22):

I think the impact that I've made, I definitely see, because I see all of those little girls cheering for me, supporting me on Instagram, on TikTok, all of the outlets. So I do understand the impact that I've made. But to fully understand the depth of it, I think it'll take one day whenever I retire, and then I look back at the career and everything that I've done to evolve the sport.

Speaker 1 (01:42):

I mean, we've seen celebrities in the stands. We've seen young gymnasts looking up at you. It's been quite a ride. I mean, it's a lot of pressure on you, I would think. How have you dealt with it?

Simone (01:53):

Yes. It's a lot of pressure, but pressure is privilege. So I'm just super excited to represent the United States every time I get the opportunity.

Speaker 1 (02:00):

There have been so many moments. One that struck me was watching three Black athletes together on the podium. When that was happening, did you

recognize the significance of it?

Simone (02:11):

Oh, absolutely. And I don't think we got to recognize it enough because the award ceremony happened so fast. They're trying to break down the set and move in the next event that's going to be there. But we all recognized it and we were really excited about it.

Summary:<|end_of_text|>

<|start_of_role|>assistant<|end_of_role|>Simone Biles talks about her expectations for the Paris 2024 Olympics, her pride in her performances, and the impact of self-care on her mental health. She also discusses the significance of being a role model for young gymnasts and the recognition of her achievements in the sport.<|end_of_text|>

Use the following code example to reproduce this output.

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
```

```
device = "cuda"
model_path = "ibm-granite/granite-4.0-h-tiny"
tokenizer = AutoTokenizer.from_pretrained(model_path)
# drop device_map if running on CPU
model = AutoModelForCausalLM.from_pretrained(model_path,
device_map=device)
model.eval()
```

```
interview_transcript = ""
Speaker 1 (00:00):
Simone, congratulations.
```

```
Simone (00:02):
Thank you.
```

```
Speaker 1 (00:02):
The last several years have been all about our expectations, what we
thought you would do, what we thought the team would do. What were your
expectations coming here?
```

```
Simone (00:10):
My expectations or at least the team's expectation was to go out there and
win gold for Team USA, and we did just that. And after that, my
expectation was go to all-around, see if you can place top three, see what
happens. And then after that, everything else was a cherry on top. So
Vault, gold. Floor, silver. I'm not mad.
```

```
Speaker 1 (00:28):
Are you proud of yourself?
```


retire, and then I look back at the career and everything that I've done to evolve the sport.

Speaker 1 (01:42):

I mean, we've seen celebrities in the stands. We've seen young gymnasts looking up at you. It's been quite a ride. I mean, it's a lot of pressure on you, I would think. How have you dealt with it?

Simone (01:53):

Yes. It's a lot of pressure, but pressure is privilege. So I'm just super excited to represent the United States every time I get the opportunity.

Speaker 1 (02:00):

There have been so many moments. One that struck me was watching three Black athletes together on the podium. When that was happening, did you recognize the significance of it?

Simone (02:11):

Oh, absolutely. And I don't think we got to recognize it enough because the award ceremony happened so fast. They're trying to break down the set and move in the next event that's going to be there. But we all recognized it and we were really excited about it.""

```
chat=[
    {"role": "user", "content": "Summarize a fragment of an interview transcript. In this interview, an NBC reporter interviews Simone Biles about her participation in Paris 2024 Olympic games.\nYour response should only include the answer. Do not provide any further explanation.\n"+interview_transcript+"\nSummary:"}
]
```

```
chat = tokenizer.apply_chat_template(chat,tokenize=False,
add_generation_prompt=True)
```

```
# tokenize the text
input_tokens = tokenizer(chat, return_tensors="pt").to(device)
# generate output tokens
output = model.generate(*input_tokens,
                        max_new_tokens=200)
# decode output tokens into text
output = tokenizer.batch_decode(output)
# print output
print(output[0])
```

Text classification

This example demonstrates a classification task for movie reviews. The user query includes classification examples to improve the model's response accuracy.

<|start_of_role|>user<|end_of_role|>Classify the sentiment of the movie reviews as positive or negative.
Your response should only include the answer. Do not provide any further explanation.

Here are some examples, complete the last one:

Review:

Oh, where do I even begin? Barbie 2023 is a tour de force that has left me utterly captivated, enchanted, and spellbound. Every moment of this cinematic marvel was nothing short of pure excellence, deserving nothing less than a perfect 10 out of 10 rating!

Sentiment:

positive

Review:

It's a shame because I love Little Women and Lady Bird. Barbie did not serve. It's a long ramble about how matriarchy is perfect and patriarchy is stupid. I agree with the latter, but the execution is just awful. Mattel's CEO and corporate people started as a mockery of men having all the top positions in a company; as the movie goes, they're just there at the back of your mind, and it's an unpleasant experience because they did nothing after. There were so many unnecessary scenes that could have been meaningful. For example, what's the scene where Allan fights other Kens for? I'm also surprised that there were no gay Barbies and Kens. And I thought this was supposed to "break" gender norms. So many seeds were planted at once, but none reached their full potential.

Sentiment:

negative

Review:

Russell Crowe's new action movie Land of Bad can't break the star's recent streak of poor Rotten Tomatoes ratings. Co-starring Liam and Luke Hemsworth, Crowe's latest action effort concerns a special ops mission in the Philippines that goes wrong, leading to a harrowing fight for survival. Crowe plays Reaper, a drone pilot who must guide a group of outnumbered and stranded soldiers back to safety.

Despite the Land of Bad cast including big names like Crowe and two of the Hemsworths, the action film has failed to impress critics, currently sitting at 52% fresh on Rotten Tomatoes on 24 reviews, becoming the fifth straight Crowe-starring film to be certified rotten by the review aggregator.

Sentiment:<|end_of_text|>

<|start_of_role|>assistant<|end_of_role|>positive<|end_of_text|>

Use the following code example to reproduce this output.

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
```

```
device = "cuda"
```

```
model_path = "ibm-granite/granite-4.0-h-tiny"
tokenizer = AutoTokenizer.from_pretrained(model_path)
# drop device_map if running on CPU
model = AutoModelForCausalLM.from_pretrained(model_path,
device_map=device)
model.eval()
```

```
prompt = """Classify the sentiment of the movie reviews as positive or
negative.
Your response should only include the answer. Do not provide any further
explanation.
```

Here are some examples, complete the last one:

Review:

Oh, where do I even begin? Barbie 2023 is a tour de force that has left me utterly captivated, enchanted, and spellbound. Every moment of this cinematic marvel was nothing short of pure excellence, deserving nothing less than a perfect 10 out of 10 rating!

Sentiment:

positive

Review:

It's a shame because I love Little Women and Lady Bird. Barbie did not serve. It's a long ramble about how patriarchy is perfect and patriarchy is stupid. I agree with the latter, but the execution is just awful. Mattel's CEO and corporate people started as a mockery of men having all the top positions in a company; as the movie goes, they're just there at the back of your mind, and it's an unpleasant experience because they did nothing after. There were so many unnecessary scenes that could have been meaningful. For example, what's the scene where Allan fights other Kens for? I'm also surprised that there were no gay Barbies and Kens. And I thought this was supposed to "break" gender norms. So many seeds were planted at once, but none reached their full potential.

Sentiment:

negative

Review:

Russell Crowe's new action movie Land of Bad can't break the star's recent streak of poor Rotten Tomatoes ratings. Co-starring Liam and Luke Hemsworth, Crowe's latest action effort concerns a special ops mission in the Philippines that goes wrong, leading to a harrowing fight for

survival. Crowe plays Reaper, a drone pilot who must guide a group of outnumbered and stranded soldiers back to safety.

Despite the Land of Bad cast including big names like Crowe and two of the Hemsworths, the action film has failed to impress critics, currently sitting at 52% fresh on Rotten Tomatoes on 24 reviews, becoming the fifth straight Crowe-starring film to be certified rotten by the review aggregator.

Sentiment:""

```
chat=[
    {"role": "user", "content": prompt}
]

chat = tokenizer.apply_chat_template(chat,tokenize=False,
add_generation_prompt=True)

# tokenize the text
input_tokens = tokenizer(chat, return_tensors="pt").to(device)
# generate output tokens
output = model.generate(*input_tokens,
                        max_new_tokens=200)
# decode output tokens into text
output = tokenizer.batch_decode(output)
# print output
print(output[0])
```

Text extraction

This example demonstrates how to extract certain information from a set of documents with a similar structural pattern.

```
<|start_of_role|>user<|end_of_role|>Extract the Line Of Credit Facility
Maximum Borrowing Capacity from the 10K sentences.
Your response should only include the answer. Do not provide any further
explanation.
```

Here are some examples, complete the last one:

10K Sentence:

The credit agreement also provides that up to \$500 million in commitments may be used for letters of credit.

Line Of Credit Facility Maximum Borrowing Capacity:

\$500M

10K Sentence:

In March 2020, we upsized the Credit Agreement by \$100 million, which matures July 2023, to \$2.525 billion.

Line Of Credit Facility Maximum Borrowing Capacity:
\$2.525B

10K Sentence:

We prepared our impairment test as of October 1, 2022 and determined that the fair values of each of our reporting units exceeded net book value by more than 50%. Among our reporting units, the narrowest difference between the calculated fair value and net book value was in our Principal Markets segment's Canada reporting unit, whose calculated fair value exceeded its net book value by 53%. Future developments related to macroeconomic factors, including increases to the discount rate used, or changes to other inputs and assumptions, including revenue growth, could reduce the fair value of this and/or other reporting units and lead to impairment. There were no goodwill impairment losses recorded for the nine months ended December 31, 2022. Cumulatively, the Company has recorded \$469 million in goodwill impairment charges within its former EMEA (\$293 million) and current United States (\$176 million) reporting units. Revolving Credit Agreement In October 2021, we entered into a \$3.15 billion multi-currency revolving credit agreement (the "Revolving Credit Agreement") for our future liquidity needs. The Revolving Credit Agreement expires, unless extended, in October 2026. Interest rates on borrowings under the Revolving Credit Agreement are based on prevailing market interest rates, plus a margin, as further described in the Revolving Credit Agreement. The total expense recorded by the Company for the Revolving Credit Agreement was not material in any of the periods presented. We may voluntarily prepay borrowings under the Revolving Credit Agreement without premium or penalty, subject to customary "breakage" costs. The Revolving Credit Agreement includes certain customary mandatory prepayment provisions. Interest on Debt Interest expense for the three and nine months ended December 31, 2022 was \$27 million and \$65 million, compared to \$18 million and \$50 million for the three and nine months ended December 31, 2021. Most of the interest for the pre-Separation period presented in the historical Consolidated Income Statement reflects the allocation of interest expense associated with debt issued by IBM from which a portion of the proceeds benefited Kyndryl.

Line Of Credit Facility Maximum Borrowing Capacity:<|end_of_text|>
<|start_of_role|>assistant<|end_of_role|>\$3.15B<|end_of_text|>

Use the following code example to reproduce this output.

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

device = "cuda"
model_path = "ibm-granite/granite-4.0-h-tiny"

tokenizer = AutoTokenizer.from_pretrained(model_path)
```

```
# drop device_map if running on CPU
```

```
model = AutoModelForCausalLM.from_pretrained(model_path,  
device_map=device)  
model.eval()
```

```
prompt = """Extract the Line Of Credit Facility Maximum Borrowing Capacity  
from the 10K sentences.
```

```
Your response should only include the answer. Do not provide any further  
explanation.
```

Here are some examples, complete the last one:

10K Sentence:

The credit agreement also provides that up to \$500 million in commitments may be used for letters of credit.

Line Of Credit Facility Maximum Borrowing Capacity:
\$500M

10K Sentence:

In March 2020, we upsized the Credit Agreement by \$100 million, which matures July 2023, to \$2.525 billion.

Line Of Credit Facility Maximum Borrowing Capacity:
\$2.525B

10K Sentence:

We prepared our impairment test as of October 1, 2022 and determined that the fair values of each of our reporting units exceeded net book value by more than 50%. Among our reporting units, the narrowest difference between the calculated fair value and net book value was in our Principal Markets segment's Canada reporting unit, whose calculated fair value exceeded its net book value by 53%. Future developments related to macroeconomic factors, including increases to the discount rate used, or changes to other inputs and assumptions, including revenue growth, could reduce the fair value of this and/or other reporting units and lead to impairment. There were no goodwill impairment losses recorded for the nine months ended December 31, 2022. Cumulatively, the Company has recorded \$469 million in goodwill impairment charges within its former EMEA (\$293 million) and current United States (\$176 million) reporting units. Revolving Credit Agreement In October 2021, we entered into a \$3.15 billion multi-currency revolving credit agreement (the "Revolving Credit Agreement") for our future liquidity needs. The Revolving Credit Agreement expires, unless extended, in October 2026. Interest rates on borrowings under the Revolving Credit Agreement are based on prevailing market interest rates, plus a margin, as further described in the Revolving Credit Agreement. The total expense recorded by the Company for the Revolving Credit Agreement was not material in any of the periods presented. We may voluntarily prepay borrowings under the Revolving Credit Agreement without premium or penalty, subject to customary "breakage" costs. The Revolving Credit Agreement includes certain customary mandatory

prepayment provisions. Interest on Debt Interest expense for the three and nine months ended December 31, 2022 was \$27 million and \$65 million, compared to \$18 million and \$50 million for the three and nine months ended December 31, 2021. Most of the interest for the pre-Separation period presented in the historical Consolidated Income Statement reflects the allocation of interest expense associated with debt issued by IBM from which a portion of the proceeds benefited Kyndryl.
Line Of Credit Facility Maximum Borrowing Capacity:""

```
chat=[
    {"role": "user", "content": prompt}
]

chat = tokenizer.apply_chat_template(chat,tokenize=False,
add_generation_prompt=True)

# tokenize the text
input_tokens = tokenizer(chat, return_tensors="pt").to(device)
# generate output tokens
output = model.generate(**input_tokens,
                        max_new_tokens=200)
# decode output tokens into text
output = tokenizer.batch_decode(output)
# print output
print(output[0])
```

Text translation

Granite 4.0 models also support tasks in multiple languages. This is a basic example of how to use the models to translate text from English to Spanish.

```
<|start_of_role|>user<|end_of_role|>Please translate the following list of
greetings from English to Spanish.
```

```
Morning!, how are things?, hello, it's good to see you, what's up?
```

```
Please only output the translation, and nothing else.<|end_of_text|>
```

```
<|start_of_role|>assistant<|end_of_role|>Buenos días!
```

```
¿Cómo estás?
```

```
¡Hola!
```

```
¡Es buenísimo verte!
```

```
¿Qué tal?<|end_of_text|>
```

Use the following code example to reproduce this output.

```
import torch
```

```

from transformers import AutoModelForCausalLM, AutoTokenizer

device = "cuda"
model_path = "ibm-granite/granite-4.0-h-tiny"

tokenizer = AutoTokenizer.from_pretrained(model_path)
# drop device_map if running on CPU
model = AutoModelForCausalLM.from_pretrained(model_path,
device_map=device)
model.eval()

prompt = """Please translate the following list of greetings from English
to Spanish.

Morning!, how are things?, hello, it's good to see you, what's up?

Please only output the translation, and nothing else."""

chat=[
    {"role": "user", "content": prompt}
]

chat = tokenizer.apply_chat_template(chat,tokenize=False,
add_generation_prompt=True)

# tokenize the text
input_tokens = tokenizer(chat, return_tensors="pt").to(device)
# generate output tokens
output = model.generate(*input_tokens,
                        max_new_tokens=200)
# decode output tokens into text
output = tokenizer.batch_decode(output)
# print output
print(output[0])

```

Granite 4.0 models support text translation as well as other multilingual tasks in English, German, Spanish, French, Japanese, Portuguese, Arabic, Czech, Italian, Korean, Dutch, and Chinese (Simplified).

Temporal reasoning

This is an example of a basic temporal reasoning question.

```

<|start_of_role|>user<|end_of_role|>Alice went to the store two days
before her birthday. If her birthday was on a Friday, what day did she go
to the store?<|end_of_text|>

```

```
<|start_of_role|>assistant<|end_of_role|>If Alice's birthday was on a
Friday and she went to the store two days before her birthday, then she
went to the store on a Wednesday.<|end_of_text|>
```

Use the following code example to reproduce this output.

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

device = "cuda"
model_path = "ibm-granite/granite-4.0-h-tiny"

tokenizer = AutoTokenizer.from_pretrained(model_path)
# drop device_map if running on CPU
model = AutoModelForCausalLM.from_pretrained(model_path,
device_map=device)
model.eval()

prompt = """"Alice went to the store two days before her birthday. If her
birthday was on a Friday, what day did she go to the store?""""

chat=[
    {"role": "user", "content": prompt}
]

chat = tokenizer.apply_chat_template(chat,tokenize=False,
add_generation_prompt=True)

# tokenize the text
input_tokens = tokenizer(chat, return_tensors="pt").to(device)
# generate output tokens
output = model.generate(**input_tokens,
                        max_new_tokens=200)
# decode output tokens into text
output = tokenizer.batch_decode(output)
# print output
print(output[0])
```

Logical reasoning

This is an example of a basic logical reasoning question.

```
<|start_of_role|>user<|end_of_role|>If both statements below are true:

If a person is a teacher, then they work at a school.

Maria does not work at a school.
```

What can we conclude about Maria?<|end_of_text|>

<|start_of_role|>assistant<|end_of_role|>We can conclude that Maria is not a teacher.

Here's the reasoning:

1. The first statement says that if a person is a teacher, then they work at a school. This can be written as: Teacher $\rightarrow$ Works at a school.
2. The second statement tells us that Maria does not work at a school. This can be written as: Not Works at a school.
3. Since the first statement establishes a conditional relationship between being a teacher and working at a school, we can use the contrapositive of this statement to draw a conclusion about Maria.
4. The contrapositive of "Teacher $\rightarrow$ Works at a school" is "Not Works at a school $\rightarrow$ Not a teacher."
5. Since Maria does not work at a school (Not Works at a school), we can conclude that Maria is not a teacher (Not a teacher).

Therefore, based on the given statements, we can conclude that Maria is not a teacher.<|end_of_text|>

Use the following code example to reproduce this output.

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

device = "cuda"
model_path = "ibm-granite/granite-4.0-h-tiny"

tokenizer = AutoTokenizer.from_pretrained(model_path)
# drop device_map if running on CPU
model = AutoModelForCausalLM.from_pretrained(model_path,
device_map=device)
model.eval()

prompt = """If both statements below are true:

If a person is a teacher, then they work at a school.

Maria does not work at a school.

What can we conclude about Maria?"""

chat=[
    {"role": "user", "content": prompt}
]
```

```

chat = tokenizer.apply_chat_template(chat,tokenize=False,
add_generation_prompt=True)

# tokenize the text
input_tokens = tokenizer(chat, return_tensors="pt").to(device)
# generate output tokens
output = model.generate(**input_tokens,
                        max_new_tokens=200)
# decode output tokens into text
output = tokenizer.batch_decode(output)
# print output
print(output[0])

```

Building prompts with transformers library

The `apply_chat_template` function from the transformers library automatically applies the basic chat template structure of Granite 4.0 models to your prompts. To build prompts that integrate advanced features via the `apply_chat_template` you must use the appropriate `kwargs`.

- Use `tools` to provide the list of tools in tool-calling tasks. This automatically activates the tool-calling system prompt and formats the tool list.
- Use `documents` to provide the list of documents for RAG prompts. This automatically activates the RAG system prompt and formats the document list.

We strongly recommend that developers use the `apply_chat_template` function to construct their prompts, as it enhances the development experience and minimizes the risk of inference errors caused by manually crafting chat templates in `Jinja`. However, you can also load a chat template that incorporates Granite 4.0's advanced features without using the `apply_chat_template` function.

In the following example, we load the template for RAG inference. Note that the final model prompt is saved in `input_text` variable. You can take this example as baseline to build wrappers for libraries that do not support `kwargs`.

```

import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

device = "cuda"
model_path = "ibm-granite/granite-4.0-h-tiny"
tokenizer = AutoTokenizer.from_pretrained(model_path)
# drop device_map if running on CPU

```



```
output = model.generate(inputs["input_ids"].to(device),
attention_mask=inputs["attention_mask"].to(device), max_new_tokens=500)
answer = tokenizer.decode(output[0][inputs["input_ids"].shape[1]:],
skip_special_tokens=True)
print(answer)
```

The execution of this script returns the assistant's output below:

```
French builders developed the Gothic architectural style from the early
12th century.
```

Inference Tips

- Verify the Max New Tokens setting. If you see a model response stopping mid-sentence, the max new tokens setting is likely set too low. This is particularly critical in long-context tasks.
- Avoid pronouns in follow-up questions. For example, do not use "Can you edit it?", instead use "Can you edit the table?".
- Reduce explanation length. If the explanations provided by the model are too long, update the instruction to make it clear there should be no additional explanations.
- Fix run-on sentences/responses. If the model is generating run-on sentences/responses, try reducing the max tokens output parameter or adding line breaks or spaces as stop token(s). This should lead the model to stop generating the output after generating these tokens.
- For prompts that include in-context examples, consider using the example labels as a stop token so the model stops generation after providing the answer.